04 Testing and evidence

Status: 20 October 2026, written against the version tagged `submission-20oct` in both repositories (the backend and frontend deployed that day). The live measurements below were taken on the backend deployed at 12:38 (`8f21177`); the tagged backend differs from it in one product file (`egoquant/agents/quant/v2/solve.py`: the reverted v2 solver shortcut, section 9), and everything else that differs is tests, documents and screenshots. Numbers are re-counted by the command next to them.

In plain words. We did not only write code, we tried hard to break it. This document lists what the automatic tests cover, how many there are, how we know they would notice a real fault (we broke the code on purpose and checked that the tests failed), what a hostile user and a crowd of simultaneous users did to the running system, and what we could *not* check. It also tells one uncomfortable story honestly: on the Windows laptops we developed on, the testing stack itself crashed at random under load (section 9). Every claim below names a file, a test or a measured number, so you can check it.

---

1. The numbers at a glance

What	Number	How it was counted
Automatic tests (offline, no network, no AI)	**3,540** selected in **141** files (3,561 collected; 21 need live market data and are deselected by `-m "not live"`)	`pytest --collect-only -q -m "not live"` on the final candidate (16 s); 3,554 after the changes described in the next row

What	Number	How it was counted
Last complete runs	**Every area in separate processes, 20 Oct (12:45 to 13:47 on the deployed backend `8f21177`; `agent_a` with its `v2` folder and `anchoring` again at 13:26 to 13:31 on the final candidate, which has the same product code as `6e217ce`; later commits are documents and screenshots)** (this Windows stack kills or stalls one long pytest process at random, section 9; `eval/run_tests_per_file.py` runs each file alone and retries a file that died or hung; no file crashed or hung): **3,538 passed, 1 failed, 1 skipped** (3,540 outcome lines, equal to the 3,540 selected tests): `agent_b` 680, `briefing` 413 passed + 1 skipped + 1 failed, `anchoring` 463, `system` 360 (incl. the 95 tests of `test_risk_game_blend.py` with the 300-investor sweep), `agent_a` 104 + 183 in `v2/`, `accounts` 265, `agent_c` 274, `risk_profile` 134, `advisor` 105, `regime` 90, `preferences` 60, the 35 top-level files 407. These per-area counts are the runner's outcome lines; the table below is the collection. They differ by one in two areas and cancel in the total: `anchoring` 463 run + 1 wallet file-permission test that cannot pass on Windows and is left out by the runner = 464 selected; `briefing` 413 passed + 1 skipped + 1 failed = 415 outcome lines against 414 selected (we did not trace that extra line). The deploy check on the live service after the price mode was set to snapshot: 71 PASS, 0 FAIL (Bideci's AI, 13:00). Earlier single complete runs: 3,188 passed, 2 failed (f9c7f98, Onur); 3,245 passed, 2 failed (`fix/bartu-7`, Bartu). **After these runs one change was made on 20 Oct:** the trade-check tool takes one size (`advisor-v11`; 13 new tests); one test was added that the downloaded report makes no request when it is opened (`tests/system/test_report_no_remote_resources.py`). 3,554 tests are selected now; the 19 test files that render the report or the seal, the trade tool and the chat ran green on the final tree (319 tests); the per-area numbers above and the table below are those of the runs before the changes	The one failure is known: a timing test of the news page (`tests/briefing/test_news_ui.py`, `test_dedupe_catches_near_identical_titles_and_scales`), not an investment rule. Bideci's AI ran the whole suite on the deployed head `8f21177`: 3,576 passed, 1 failed. That head still carried 77 more tests than the final candidate (the 78 tests of the v2 shortcut that was reverted later, section 9, minus the one guard test added), which is why its number is higher than the 3,540 above. The failure was the Windows-recorded golden hash of the flag-off regime test, which depends on the processor and the math-library threads: it was recorded on the development PC, passes there and fails on that other machine.
Tests by area	see the table below	same collect run

Area (folder under `tests/`)	Tests	What it covers
`agent_b`	680	Behaviour (trade history, IBKR file parser incl. hostile files, game and crash-test evidence)
`briefing`	414	Agent D: news items, tone labels, advice filter, jobs
`anchoring`	464	On-chain seal: canonical record, fingerprint, sealed report and seal file, routes, seal v2 attacks, every committed proof against a snapshot of the real transactions
top level (`tests/*.py`)	407	Routes, request-size limits, chat, deploy check, price fallback drills, plain wording
`system`	360	Whole chain on synthetic prices: pipeline, Risk Engine, report, goal simulation, what-if, trade check, Risk Game blend
`agent_a`	287	The optimiser and the backtest (104), and the optional v2 method (183 in `v2/`)
`accounts`	265	Sign-in separation, saved reports, trade check for accounts, hostile inputs, chat trade tool
`agent_c`	274	Universe choice, price sources, snapshots, caches, fallbacks
`risk_profile`	134	Questionnaire, scoring, C1 to C5 table
`regime`	90	Market-phase sampler and its fall-back
`advisor`	105	Advice guard, trade explanation, AI-down behaviour

Area (folder under `tests/`)	Tests	What it covers
`preferences`	60	The investment-preference form

2. How to re-run everything

```
pip install -e ".[dev]"
pytest -q -m "not live"                      # everything offline (a long run; on Windows see section 9)
pytest -q tests/agent_c                      # one area
RISK_GAME_SWEEP_N=300  pytest -q tests/system/test_risk_game_blend.py        # the 300-investor sweep (default)
RISK_GAME_E2E_N=100    pytest -q tests/system/test_risk_game_end_to_end.py   # 100 investors through /portfolio (default 30)
CONCURRENT_ROUNDS=3    pytest -q tests/system/test_concurrent_builds.py      # 3 rounds of 8 simultaneous builds (default 1)
TRADE_SWEEP_N=1000     pytest -q tests/system/test_trade_check_sweep.py      # 5 personas x 1,000 random trades
python eval/deploy_check.py https://web-production-59960.up.railway.app --origin https://www.egoquant.app --quick   # the live system (read-only)
```

The stress tools of section 7 and 8 are in `eval/stress/` (a README there says how to run each; the load and edge tools start their own local server, use the committed price snapshot, and need no network and no key; the two probe scripts, `probe.py` and `probe_after_deploy.py`, call a deployed URL instead and only send read-only requests). **On Windows** run the suite file by file with a retry (see section 9): a single native crash of the Python process would otherwise lose the whole run.

```
python eval/run_tests_per_file.py . results.txt --dir tests/system      # one pytest process per file; a file that dies or hangs is retried up to three times
```

3. What the tests check, by kind

3.1 Rules with an independent answer key (oracles). The Risk Game blend (`docs/RISK_GAME_BLEND.md`): the category is the lower of the capacity cap and the blended tolerance, the no-loss gate forces C1. `tests/system/test_risk_game_blend.py` (95 tests) writes the rule again from the contract and compares it with the real endpoints for a **300-investor random sweep** (answers drawn from two latent dials, boldness and capacity, plus a 1-in-14 "no loss at all" gate; it asserts that all five classes, the gate, and a game that lowers, raises or is stopped by the cap occur). It also covers 33 hostile `risk_game` inputs, NaN and Infinity as raw JSON, old and future-dated results, a game sent in two places, 14 weight settings and 10 "may raise" settings, and the sealed chat, what-if and trade contexts.
`tests/system/test_risk_game_end_to_end.py` (3 tests; 100 investors with `RISK_GAME_E2E_N=100`) builds full portfolios and checks: approved, optimiser optimal, `c_level` equal to the profile and to the rule, `report.category_basis` present exactly when a game was sent and carrying the profile's own numbers, weights sum to 1, nothing above the class on offer, sealed contexts present, and a rebuild without the game gives the questionnaire class. `tests/accounts/test_account_risk_game.py` (10) does the same for saved reports, the replay rule, delete-my-data and the on-chain record (with and without a game, confirm to proof to verify, and a tampered record does not verify).

3.2 Property sweeps (random inputs, rules that must always hold). - Trade check: `tests/system/test_trade_check_sweep.py`, 5 personas times random trades (80 each by default). Properties: same input twice gives the same answer; order of holdings does not matter; "advisable" implies an independent Risk Engine run approves; selling never raises the stock share; buying a product the investor may not hold is never advisable; odd amounts (1e-12 to 1e12, the whole account) raise nothing but the plain refusal. One long run, 5 x 1,000 trades, held every property (Damla, 20 Oct, note in `overnight/damla.md`). The sweep found and fixed a real hole: a tiny buy of a product above the investor's risk level was only a warning (`_buy_gate` now refuses it whatever the amount). - Engine edge cases:
`tests/system/test_engine_edge_cases.py` (3 tests, about 80 cases). Extreme but valid inputs through

`/portfolio`: 1 USD and 1e12, "needed" equal to "invested", horizons 0.1 and 60 years, goals below the start amount, 1e12 and a goal of 1 USD with a monthly flow of +1e9, monthly flows of plus and minus 1e9, liquidity and cash buffer 1.0, 1 and 20 interest areas, holdings that sum to 0, 0.5 or 1.5, holdings that are not in the catalog, an IBKR report with an unknown ticker. Each is a plain 200 or a plain 422, never a 5xx; every number finite; weights not negative and summing to 1; the verdict (`approved`, violations, checks, the report's own verdict) consistent; the class equal to `/profile`'s; an approved portfolio inside its class caps. - Hostile inputs for accounts: `tests/accounts/test_extreme_inputs.py` (80 cases: oversized bodies, unicode and null bytes, NaN and Infinity, forged and oversized tokens, path-like ids, formula cells); `tests/test_no_echo_422.py` and `tests/test_validation_messages.py` (a refused value is never repeated in the answer); `tests/test_body_limit.py` (16: 413 for a declared length over the limit, 411 for a chunked body). - HTTP surface: `tests/test_http_edges.py` (5 tests): the four security headers on every error status (404, 405, 411, 413, 422, 401, 429), CORS allows only the Vercel origin and never credentials, every account route answers an anonymous caller with a plain 401 or 422, no odd method or path ends in a 5xx.

3.3 Same answer, whoever else is using the system. `tests/system/test_concurrent_builds.py` (3 tests): eight different investors (every class, goals and none, an IBKR file, holdings, a Risk Game, different interests) are built alone and then all at once in threads; every answer must equal the one built alone, with the sealed chat, what-if and trade contexts opened and compared by their facts; the same for `/trade-check`, `/profile` and `/preferences/form` running beside the builds, and for six what-if runs (including re-optimise) in parallel. `tests/test_build_gate.py` (7) pins the build limit of section 7.

3.4 Agent C (market data). `tests/agent_c` (274): universe choice and caps (`test_market_intel.py` 90, `test_universe_v2.py` 26), the directory of listed products (`test_directory.py` 65), snapshots and their labels (`test_price_snapshot.py` 28, `test_yahoo_source.py` 8), the slow-feed and snapshot-only modes (`test_slow_feed_and_snapshot_mode.py` 12, `tests/test_slow_feed_drill.py` 18), cache warm-up and write failures (`test_warm_cache.py` 11, `test_cache_write_failure.py` 6), fetch time limits and fallbacks (`test_fetch_time_limit.py`, `test_data_fallback.py`), the per-build memo of snapshot reads (`test_snapshot_memo.py` 11). The report-level checks are in `tests/system/test_snapshot_banner.py` and `tests/test_price_cause_labels.py`.

---

4. Do the tests notice a real fault? Mutation checks

A test that cannot fail proves nothing. After writing each group we **broke the code on purpose** (one change at a time), ran the group, and confirmed that it failed; then we restored the code. All were done by hand in a separate copy of the repository; none of the broken code is in the product.

Group (file)	What we broke	Result
Risk Game rule (`tests/system/test_risk_game_blend.py`)	7 changes in `egoquant/risk_profile/scoring.py` (e.g. the game still counts at weight 0; rounding; the cap ignored)	7 of 7 caught (one needed a stronger assertion: the whole `/profile` answer at weight 0 must equal "no game")
Account and on-chain paths (`tests/accounts/test_account_risk_game.py`)	5 changes in the routes and the record builder	5 of 5 caught
100-investor end to end (`test_risk_game_end_to_end.py`)	the basis shown without a game; a wrong final class; two levels swapped	3 of 3 caught
Concurrent builds (`test_concurrent_builds.py`)	a shared global written at the start of a build and read at the end	caught: 6 of 8 investors received another investor's class

Group (file)	What we broke	Result
Engine edge cases (`test_engine_edge_cases.py`)	a crash injected for horizons under 0.2 years	caught as an HTTP 500
Build gate (`test_build_gate.py`)	slot never given back; given back only on success; no gate; slot taken before validation	4 of 4 caught (5, 5, 4 and 3 tests failed)
Price note (`tests/system/test_price_note_in_reports.py`)	the note built from the whole file instead of the priced instruments	caught (2 of 3 route tests failed)

One honest remark: the first version of the concurrency mutation (a shared global read a few microseconds after it was written) was **not** caught. A test with threads cannot see a race that short. The version above, where the value is read at the end of a multi-second build, is caught. So the concurrency tests prove "no mixing between builds", not "no data race in any line of code"; see section 9.

---

5. The hostile pass (20 October, from the release `a18a157`)

In plain words. We played a hostile and a very busy user against our own system, first on a local copy (no network, committed price snapshot), then on the live system with a hard budget of 120 read-only requests (93 used in the first pass and 111 in all after the re-check below; no portfolio was built and no account was touched). Method and scripts: `eval/stress/`. Results, with what happened to each finding:

Id	Sev	Finding	Status at 00c93c8
H1	A	Memory and waiting time grow with the number of **simultaneous** goal builds; nothing capped them (section 7)	Fixed: build gate, default 1 slot (`fix/damla-12`, 59eb912)
H2	C	Under load the 5-second in-request news briefing is dropped; the report is the same without it	By design (documented in `egoquant_routes.py`)
H3	C	`briefing_context.interest_tickers` has 115 or 1,260 entries depending on whether the background directory load has finished	By design
H4	B	A C3 to C5 investor with a cash need of 50% or more gets no portfolio ("could not build a portfolio") and the banner called it "rejected by the Risk Engine"	Wording fixed (28507a9); the dead end for cash needs of 50% or more remains open
H5	B	`GET /health/` answered 307 to **http**:// (a downgrade)	Fixed (28507a9: redirects off, proxy headers)
H6	C	`POST /debug/portfolio` with an invalid body was a 422, not a 404 (the route's existence showed)	Fixed (28507a9)
H7	C	An anonymous `POST /account/portfolio`, `/account/trade-check`, `PUT /account/onboarding` gives 422 before 401	Open, accepted (the schema is public in `/openapi.json` anyway)
H8	C	A garbage bearer token gave 403 "Not allowed."	Fixed: 401 (28507a9)
H9	C	No rate limit on `/profile`, `/preferences/*`, `/questionnaire`, `/verify`, `/briefing/*`; the `/portfolio` bucket (20 a minute) is shared by all visitors; the 429 had no `Retry-After`	`Retry-After` fixed (28507a9); the rest open, accepted (documented in `docs/SECURITY.md` T11)
H10	C	`/docs`, `/redoc`, `/openapi.json` and `/data/status` are public	Open, deliberate for the judges; `/data/status` shows no key or fingerprint (`tests/test_no_secrets_in_public_endpoints.py`)

Id	Sev	Finding	Status at 00c93c8
H11	C	`HEAD /health` and `OPTIONS /health` answer 405	Open, harmless
H12	C	No HSTS header from the Railway domain	Fixed (28507a9)

What held (live, 93 requests): 13 distinct values planted in refused requests were never repeated in any answer; every 404, 405, 411, 413, 422 and 401 carried all four security headers; CORS allowed only the Vercel origin, no credentials; every `GET /account/*` and `/what-if` without a token was a plain 401; `/verify` with garbage was a plain 422; no 5xx and no stack trace anywhere. **Locally:** 129 extreme valid inputs gave 0 server errors and 0 non-finite numbers.

Live re-check after the deploy (20 Oct, after the 12:38 deployment of the merged backend `8f21177`). `eval/stress/probe_after_deploy.py` sent 18 read-only requests (no portfolio build, no account, no AI call), so 111 of the 120 allowed requests are used in all. **All 18 checks pass.** They ran while the price mode was still `live`; Bideci's AI switched it to `snapshot` at 13:00, after them (the 71 checks of section 1 ran after the switch). `/data/status` shows the library versions and the price mode; `/health/` is a plain 404, not a redirect; the HSTS header is on; `/debug/portfolio` is a 404 even for an empty body; a garbage token and no token are both a plain 401 (it was a 403 before); a refused body is a plain 422 in sentences and none of the planted values came back, on `/profile`, `/portfolio`, `/trade-check` and `/verify`; a declared 1.1 MB body is a 413 and a chunked body a 411; CORS allows only the Vercel origin and refuses another one; 404 and 405 answers carry all four security headers; `/what-if` without a token is a 401. No 5xx and no stack trace in any answer. Slowest answer 1.9 s (the garbage token), all others 0.7 s or less. **Not re-checked live:** the 503 "busy" answer of the build gate and the `Retry-After` of a 429 (both need many builds or many requests at once, which we do not run against the live system); they are covered by tests on a local copy (section 7).

---

6. Earlier evidence from the data and optimiser lane (Damla)

- Snapshot resilience: the committed price snapshots are complete (112 of 112 Tiingo tickers, 292 of 292 Yahoo tickers, `tests/agent_c/test_price_snapshot.py`); a truncated or damaged snapshot file used to give an HTTP 500 and now becomes a clean "market data unavailable" (`test_a_damaged_snapshot_file_is_a_source_error_never_a_crash`); a cache directory that cannot be written no longer breaks a build (`tests/agent_c/test_cache_write_failure.py`).
- Unreachable network: Agent C used to try every candidate of every cluster (191 s for one build); two failures in a row now switch the whole run to the snapshot (about 10 s). Slow vendor: a vendor at 8 s per request used to cost 249 s; builds now have a time budget (`tests/agent_c/test_fetch_time_limit.py`, `tests/test_slow_feed_drill.py`, `tests/agent_c/test_slow_feed_and_snapshot_mode.py`).
- Provenance: after a snapshot fallback four places carried the wrong date (the advisor's `prices_as_of`, the report header, the trade-check notes, the account summary); they now carry the snapshot's date and provider (`tests/system/test_snapshot_banner.py`, `tests/accounts/test_saved_summary_prices.py`).
- Memory of the goal simulation (measured 19 and 20 Oct, Windows working set including about 145 MB of imports): one heavy goal build alone peaks at 298 to 415 MB with a 10-year goal and up to 875 MB with a 30-year goal, and each extra simultaneous build adds about 150 MB (measured with 60-year goals, section 7). A 30-year build on the 1 GB Railway box has **not** been measured live. This is why the build gate exists (section 7). The cause is in Agent A's goal simulation (it keeps the full path array of every shortlisted candidate); the gate bounds the damage, the code itself was not changed.

---

7. Many users at once: the build gate

In plain words. One server process builds portfolios mostly on one processor core (we saw this on the development PC; it was not measured on the 2-core Railway box). Ten builds at once did not finish faster than ten builds one after another in our runs, but they use ten times the memory, and the web host gives up on any answer slower than about four minutes. We measured this, found that the system could be pushed over by 20 requests inside its own rate limit, and added a limit: only **one** portfolio is built at a time (`EGOQUANT_MAX_CONCURRENT_BUILDS`, default 1); the others wait their turn. **Through the website** the wait ends after 115 seconds with the website's own "too slow" answer (a 504; `BUILD_PROXY_TIMEOUT_MS` in the frontend's `src/lib/build-timing.ts`). **A direct call to the API** waits up to 120 seconds (`EGOQUANT_BUILD_WAIT_SECONDS`) and then gets a plain 503 "The service is busy building other portfolios. Please try again in a minute." with `Retry-After: 30`. So a visitor of the website meets the 504 wording, not the 503 one, and what the backend does with a request the website has already given up on was not tested.

Test: 20 identical heavy goal builds started at the same instant (60-year horizon, goal, 8 interest areas), local server on the committed snapshot, 3 runs per row unless stated (`eval/stress/conc.py ... flood`):

Setting	Peak memory	Result
no limit, 2 / 4 / 8 / 20 at once	461 / 748 / 1,364 / 3,171 MB	20 at once: each build took 206 to 239 s in the first run; in the next three runs the server process was killed (3 of 4 runs in all)
limit 3	586 to 614 MB	when it finished: 14 served and 6 turned away at 120 s (wall 130 s); in 3 of 6 runs three builds hung for more than 5 minutes (section 9); with one math thread forced, 3 of 3 finished
limit 2	stable with one BLAS thread (3 of 3); without it one run took 360 s	
limit 1 (chosen)	about 300 MB	**First measurement (`a18a157` plus the limit, quiet PC): all 20 served, 88 to 89 s in total**, `/health` slowest answer 2.7 s (3 of 3); with one BLAS thread also 3 of 3. **Repeated on the final code (`8f21177`, one math thread, PC busy with other work, one run each): 11 served and 9 turned away at 120 s** (wall 125 s, peak 335 MB, `/health` slowest 2.2 s); the same code without Agent C's price memory: 8 served, 12 turned away (wall 134 s, peak 332 MB). No crash and no hang in either run. A single heavy build alone took 12 to 17 s on this PC against 6 to 13 s on `a18a157` (same busy PC; the cause is not isolated), so fewer builds fit into the 120 s wait.

The Railway box has 1 GB and 2 cores (read by Bideci's AI), so "about 150 MB per extra build" decides it. A second slot bought no speed on the development PC (limit 3 was not faster than limit 1: 130 s with 6 turned away against 89 s with all served); whether a second slot would help on the 2-core Railway box was **not measured**. It costs memory and exposure, so the default is 1. The limit applies to `/portfolio`, `/account/portfolio`, `/trade-check`, `/account/trade-check`, the chat's trade tool and the what-if re-optimise (all go through `_execute` in `egoquant_routes.py`). Tests: `tests/test_build_gate.py` (7). Cost of the limit: when many people press "build" in the same minute, the last of them waits up to 115 seconds on the website (120 seconds on a direct API call) and then sees "too slow" (website) or "busy, try again" (API); it never crashed the service in our runs. In the worst local test above (20 of the heaviest 60-year goal builds in the same second) 11 were served and 9 turned away on the final code (8 and 12 without Agent C's price memory). Ordinary builds are lighter (under 7 s each on saved prices in the deploy check), but a flood of them was **not tested**: 20 at the same moment would need about 100 to 140 s, which is about the length of the wait. The app's own rate limit lets through 20 builds a minute for all visitors together.

8. A crash we could reproduce without our code

In plain words. While measuring section 7 the server process sometimes died, with no error message of ours. A colleague's full test run died the same way ("Windows fatal exception: access violation", in a pandas function, while another thread was copying a table). We reduced it to 40 lines that use only the price table and pandas: 12 threads read one shared price table, hash it and copy it (`eval/stress/snaprace.py`).

Python libraries (this PC: Python 3.13.5, Windows 11)	Runs	Process crashes (segmentation fault)
pandas 3.0.6 + numpy 2.5.4 (what is installed here), plain shared table	12	2
same, with a lock around every read of the shared table	3	1
same, with every lazy index built before sharing	3	1
pandas 2.3.3 + numpy 2.2.6 (installed beside, not replacing)	9	0

A run counts as a crash only if the process died with a segmentation fault; a run that was merely too slow under the load of its neighbours and hit our own time limit is not counted.

Reading: the lock and the index warm-up do **not** remove the crash, so it is not caused by sharing one table; pandas 3 with numpy 2.5 on this Windows machine is not safe for concurrent work. Pandas 2.3.3 had no crash in 9 runs against 4 in 18 for pandas 3 (all rows), which is suggestive but too few runs to conclude. We therefore did **not** switch to the older libraries (a missing wheel for the host's Python 3.14 would have broken the deployment at the worst moment). Instead `requirements.txt` **pins every package** (56 exact `==` versions, none left open; among them numpy 2.5.4, pandas 3.0.6, scipy 1.18.1, requests 2.34.2) so that a later release cannot change what runs. Protection in the product: one build at a time (section 7) and one math thread per process (`main.py` sets `OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS`, `MKL_NUM_THREADS` to 1 before numpy is imported).

Library versions. Development PC: Python 3.13.5, pandas 3.0.6, numpy 2.5.4, scipy 1.18.1. Live server: Python 3.14.4 (`GET /data/status`). *Checked live (20 Oct, after the 12:38 deployment):* the `libraries` block of `GET /data/status` reads **pandas 3.0.6, numpy 2.5.4, scipy 1.18.1** on **Python 3.14.4**, which are the same pandas, numpy and scipy versions as on the development PC (only the Python minor version and the operating system differ). What this means: the library versions in which we reproduced the crash on Windows are the versions that run in production on Linux. We could not test on Linux (no Linux machine was available), so we do not know whether the crash exists there; the protections of section 7 (one build at a time, one math thread) are in place for that reason, and the pins make the tested stack the running stack.

9. What the Windows test stack taught us

In plain words. The product runs on Linux in the cloud. We developed and tested on Windows laptops with the newest versions of Python's numerical libraries, and that combination was less reliable than our code. We say so because a judge who reruns the tests on Windows may see it.

What happened, in order: 1. One test process of the lead integrator spun a processor core for 4.5 hours (15,308 CPU seconds) and had to be killed. Under a debugger the main thread sat in `threading.Thread.start()` waiting for a new thread that never began running, while one native thread without any Python code spun. 2. Our own flood of simultaneous builds killed the server process in 3 of 4 runs without the build limit, and left three builds frozen in `Thread.start()` in 3 of 6 runs with a limit of 3.

The same freeze (a thread that never starts) was seen with only the standard library in play. 3. Full test runs died at random with "access violation": once inside `tests/system/test_concurrent_builds.py` (pandas hashing while another thread consolidated a table) and, with one math thread forced, in the thread that the web test client creates for each request. 4. On the candidate tree (1593ed4) the concurrency, build-limit and HTTP tests (15 tests) passed 3 times out of 3 (2 min 18 s to 2 min 39 s); in one of the three runs Python's fault handler printed "Windows fatal exception: access violation" with a thread inside the linear-programming solver of scipy on the stack, and the run still finished and passed. On Windows that handler also reports first-chance exceptions that a library handles itself, so the message alone does not prove a crash. 5. A standalone test of 3,602 thread starts beside busy numerical threads showed no problem (`eval/stress/`, worst start 0.22 s), so we could not isolate the cause in the time available. Candidates: the thread-start path on Windows (loader lock or native thread pool), and pandas 3 / numpy 2.5 on this platform (section 8).

What we did about it: one math thread per process; one build at a time; the suite is run **per file with an automatic retry** (`eval/run_tests_per_file.py`: it retries a file whose process died or hung for 900 seconds) so one native crash loses one file, not the run; and we report it here instead of hiding it. What it does **not** show: any wrong investment number. Every comparison of results (sections 3.3 and 7) found identical weights, classes, hashes and goal chances whenever a build finished.

A numerical lesson from the same day (20 Oct). The optional portfolio method v2 (off by default; the live service runs v1 and was never affected) had a shortcut that returned the target portfolio itself whenever it already met every limit. That is not what the solver returns: on the balanced demo investor the target met every limit, but the solver (SLSQP) stopped 0.028 away from it and still reported "optimal", and the shortcut moved 3 of the 12 blocks of the v2 demo pack (`docs/demo_data/demo_numbers_v2.json`). We reverted it, so the solver decides the weights again, and added a guard test that fails if the shortcut comes back (`tests/agent_a/v2/test_solve_level_uses_the_solver.py`). Even so, the v2 results can differ a little between computers, because a different processor or number of math-library threads can end on a near-equal portfolio (`06_LIMITATIONS_AND_ROADMAP.md` section 2.3).

---

10. What is not covered

- **Live AI behaviour.** The advisor and trade-explanation prompts have red-team suites (`eval/chat_redteam.py`: 24 of 24 on `advisor-v5`; `eval/trade_explain_redteam.py`: 51 of 51 on `trade-explain-v3`, twice); on 20 Oct the current versions were run live: `trade-explain-v4` 51 of 51; `advisor-v10` 34 of 42 (all 24 non-trade cases passed; 8 of 18 trade cases failed safely for one cause, the trade tool's two size fields, which the model always both filled), fixed as `advisor-v11` (one size field) and re-run: 41 of 42 (24 of 24 non-trade, 17 of 18 trade; the miss is a false positive of the keyword check on a correct reply); raw files `eval/results/chat_redteam_20261020_*.jsonl` and `trade_explain_redteam_20261020_162406.jsonl`. All offline tests mock the AI.
- **Real browser clicks and real Supabase.** The account isolation tests use a fake store that enforces the same rules as the SQL policies; the SQL proofs and the live smoke test are the database owner's (`supabase/tests/`, `docs/supabase/RUN_LIVE_SMOKE.md`).
- **A real cold start on the host**, the real download of the full product directory (offline we use the committed catalog), and a real Binance block from a US server.
- **The Windows crash cause** (section 9).
- **Statistical backtests of the investment method** are in `eval/results/` (regime model) and described in `03_METHODS_AND_MODELS.md`; this document does not repeat them.